

How Safe Are Agent Safety Monitors? —Three Layers of Over-Optimism in Authorization-Conditioned Effect Monitoring—

May 22, 2026

Abstract

Tool-augmented LLM agents should be monitored by what their tool calls actually do relative to the authorized task, not only by tool names or call syntax. We introduce AUTH-SAFEINV, an action-level evaluation target for authorization-conditioned realized-effect monitoring, and show that current safety evaluation can be over-optimistic at three layers. **Layer 1 (tool-surface):** linear probes trained on Qwen3-8B representations detect causal effects within seen tools, but can fail under leave-one-tool-out coverage stress tests with held-out false negative rates up to 0.80, and lexical normalization does not remove the failure. **Layer 2 (trace-label):** execution-trace verifiers that achieve FNR/FPR of 0.0/0.0 with full structured labels degrade substantially when label fields are hidden (mean FNR 0.0353 $\rightarrow$ 0.1162) or reduced to minimal evidence (mean FNR 0.1465). **Layer 3 (row-to-action):** row-level 0/0 FNR/FPR can coexist with action-level false denial rates of 1.0 and all-allow calibration collapse under simple thresholding. Pure representation-side repair (contrastive projection, schema conditioning, decomposed verification) fails to beat strong baselines under strict held-out protocols—three gate-failure results we report as important negative evidence. A verifier-assisted execution monitor achieves controlled-setting FNR/FPR of 0.0/0.0 across protocol and browser-runtime traces, but its action-level limits and the competitive performance of a raw-status boundary baseline mean it is a constructive framework with honest limits, not a deployed safety solution. We provide a unified evaluation framework, three-layer diagnostic evidence, and honest gate-failure reporting as a foundation for rigorous agent safety evaluation.

1 Introduction

LLM-based agents increasingly act by invoking tools: reading and writing files, calling web APIs, sending messages, executing shell commands, and browsing the web. The safety question is therefore not only whether a tool call is syntactically

valid, but whether the causal effects of that call are justified by the current task. A request to summarize a local file may authorize reading file contents; it does not by itself authorize network egress, file deletion, or sending the contents elsewhere.

This framing exposes an evaluation gap behind many agent defenses. Boundary and behavioral mechanisms such as causal attribution, task-specific access control, provenance tracking, and architectural isolation [3, 4, 5, 6] ultimately rely on some account of *what a proposed action would do*. Recent work on effect-trace semantics [8], provenance-aware capability contracts [7], and runtime interception [10] has begun to formalize this requirement. If the same downstream effect is represented differently depending on whether it is produced by a direct API call or by an equivalent shell command, a learned safety monitor can appear reliable on covered tools while failing on an unseen surface form. Conversely, if the monitor uses structured trace labels that are close to the answer, it can appear stronger than it actually is. And if monitor evaluation reports only candidate-effect row metrics without aggregating to action-level allow/deny decisions, it can overstate real safety.

We study these three layers of over-optimism in a controlled setting, introducing AUTH-SAFEINV as an action-level evaluation target. Given a task context c , the authorized effect envelope $A(c)$ specifies which causal effects are permitted. A tool call (a, S) produces realized effects $\Omega(a, S)$. Safety holds when $\Omega(a, S) \subseteq A(c)$; otherwise $U(c, a, S) = \Omega(a, S) \setminus A(c)$ constitutes unauthorized effects. Our evaluation target asks: can a monitor reliably detect when $U(c, a, S) \neq \emptyset$, and does its evaluation at each layer (tool-surface, trace-label, action-level) honestly reflect its real performance?

We make the following contributions:

1. **Auth-SafeInv: an action-level evaluation target.** We formalize authorization-conditioned realized-effect monitoring as an evaluation object with explicit definitions of $A(c)$, $\Omega(a, S)$, $U(c, a, S)$, and safety-centric metrics (unauthorized-effect FNR, authorized-effect FPR, ToolProxyGap). This is distinct from benchmark-based evaluation (which measures attack success or task completion) and from provenance-based defense (which traces why a call occurred).
2. **Surface-form fragmentation diagnosis.** Using linear probes over Qwen3-8B representations and leave-one-tool-out (LOTO) splits, we show that the same causal effect can be detected within a seen tool surface but missed on a held-out tool—a representation-level failure we term *surface-form fragmentation*. This is not a lexical shortcut: lexical normalization leaves overall FNR essentially unchanged. Probe-mediated interchange intervention (pIIA) provides convergent activation-level evidence. This is, to our knowledge, the first systematic representation-level diagnosis of cross-tool effect invariance in LLM agents.
3. **Three-layer over-optimism evidence.** We provide controlled diagnostic evidence that safety evaluation overstates monitor performance at three

layers:

- **Tool-surface layer:** LOTO stress tests reveal held-out FNR up to 0.80, and no simple baseline (pooling, reweighting, tool-conditioning, Procrustes alignment) eliminates the failure.
 - **Trace-label layer:** execution verifiers that use full structured trace labels achieve low FNR, but label-hidden and minimal-evidence views degrade by $3.3\times$ – $4.2\times$ in mean FNR.
 - **Action-level layer:** row-level FNR/FPR of 0.0/0.0 can coexist with action-level false denial rates of 1.0, and simple validation-calibrated thresholding collapses to all-allow on several trace families.
4. **Constructive framework with honest gate-failure reporting.** We report three negative results where pure representation-side repair (contrastive projection, schema conditioning, decomposed verification) fails to beat strong baselines under strict held-out protocols. We provide a verifier-assisted execution monitoring framework (EffectVerif-AuthMonitor) that separates effect presence verification from authorization judgment. This framework achieves controlled-setting FNR/FPR of 0.0/0.0 on live protocol and browser-runtime traces, but we document its honest limits: static verifiers fail on provider and browser effects, action-level calibration remains unsolved, and a handcrafted raw-status boundary baseline is competitive. We release all artifacts as an open-source evaluation suite.

Through experiments spanning 459 original diagnostic scenarios, an expanded 1,464-row Auth-SafeInv counterfactual suite, and 8,988 trace-based candidate-effect rows across controlled sandbox, live protocol, and file-backed headless Chrome browser-runtime traces, we provide a unified evaluation framework for a problem that, we argue, the field needs to measure before it can solve.

2 Related Work

2.1 Agent Safety Benchmarks and Runtime Defenses

Recent agent safety research has systematically mapped the attack surface [13, 14] and demonstrated real-world exploitability [1, 2, 15]. Large-scale benchmarks evaluate complementary objects: AgentDojo [16] evaluates prompt-injection defenses; ToolEmu [17] uses LM-emulated sandboxes; AgentHarm [18] measures harmful task completion; ASB [19] broadens evaluation across scenarios, tools, attacks, and defenses; ST-WebAgentBench [20] evaluates safety-policy compliance in web agents; TraceSafe [21] assesses mid-trajectory guardrail safety. Our work is not another benchmark for attack success or task completion. It asks whether a monitor can detect *unauthorized realized effects* consistently across tool surfaces, and whether common evaluation practices overstate that ability.

Table 1: Novelty boundary relative to agent security benchmarks, runtime defenses, and effect-level monitoring systems. The key distinction is that we provide an evaluation target and three-layer over-optimism diagnosis, not an **enforcement mechanism**.

Work type	Representative work	Primary judgment object	Difference from this paper
Prompt-injection benchmark	AgentDojo, ASB [16, 19]	attack success / utility	unauthorized-effect FNR under tool-surface variation
Risk discovery	ToolEmu [17]	risky tool outcomes in emulation	monitor invariance for realized effect labels
Harmful-task benchmark	AgentHarm [18]	harmful task completion	task-authorized effect consistency
Trajectory safety	TraceSafe, ST-WebAgentBench [21, 20]	mid-trajectory guardrail accuracy	three-layer evaluation framework
Causal attribution defense	AttriGuard, CausalArmor [3, 22]	why a privileged call occurs	what effect occurred and whether authorized
Boundary enforcement	ClawGuard [4]	pre-action constraint satisfaction	execution evidence after/during realization
Provenance auditing	ARGUS [5]	trusted influence over decisions	realized-effect verification plus authorization
Provenance contracts	PACT [7]	argument-level authority binding	effect-level probe evaluation across surfaces
Formal effect semantics	Alignment Contracts [8]	effect-trace satisfaction proofs	representation-level diagnosis, not formal methods
Causality tracking	ARM [9]	transitive denial influence	cross-tool representation invariance
Runtime interception	AgentTrust, Parallax [10, 11]	pre-execution allow/block	post-execution effect verification framework
Evidence certificates	ECA [12]	hallucination-to-action gate	effect-readout invariance, not perceptual certification

Defense mechanisms operate primarily at the behavioral or boundary layer. AttriGuard [3] performs causal attribution by re-executing agents under attenuated context views. CausalArmor [22] uses causal ablation at privileged action points. ClawGuard [4] derives task-specific access constraints before tool invocation. ARGUS [5] constructs influence provenance graphs. XOA [6] proposes architectural sandboxing. These methods rely, explicitly or implicitly, on some account of what a tool call would or did do. Our work evaluates a monitor-level target orthogonal to these defenses: whether the realized effect set exceeds the task-authorized envelope, and whether current evaluation practices honestly reflect monitor performance.

A rapidly growing body of 2026 work explicitly targets effect-level and provenance-level monitoring. PACT [7] enforces provenance-aware capability contracts at the argument level. Alignment Contracts [8] formalize effect-trace semantics with Lean 4 proofs. ARM [9] detects causality laundering through provenance graphs augmented with counterfactual edges. AgentTrust [10] provides runtime tool-call interception with allow/warn/block/review verdicts. Parallax [11] enforces cognitive-executive separation with adversarial validation. ECA [12] uses evidence-carrying certificates to prevent hallucination-to-action conversion. These systems focus on *enforcement* and *prevention*. Our work is complementary: we focus on *evaluation*—how to measure whether a monitor (of any architecture) can reliably detect unauthorized effects, and what evaluation practices make that measurement over-optimistic.

2.2 Causal Abstraction and Representation Diagnosis

Causal abstraction [25, 27] provides the theoretical foundation for verifying whether a neural network implements a high-level causal model. Sutter et al. [28] proved that without constraining the alignment mapping τ to be low-complexity, causal abstraction becomes vacuous. We adopt linear probes as the alignment class and use probe-mediated interchange intervention (pIIA) as a diagnostic for cross-form effect-direction transfer, not as an SCM-based causal proof.

2.3 Contrastive Representation Learning and Domain Generalization

Our surface-form fragmentation diagnosis connects to domain generalization [23, 24, 29, 30], where tool names and call styles act like domains while the safety-relevant target is the downstream effect. The difference is our safety metric and evaluation target: we care primarily about unauthorized-effect false negatives under held-out tool coverage, not average accuracy.

3 Formal Framework

3.1 Safety System Model

We model an authorization-conditioned agent safety monitor as a triple $\mathcal{S}_c = (\Phi, \Psi_c, \Omega)$, where:

- $\Phi = \{\tau_1, \dots, \tau_k\}$ maps an action’s LLM embedding to predicted effect probabilities. In our linear-probe instantiation, $\tau_i(\mathbf{h}) = \sigma(\mathbf{w}_i^\top \mathbf{h} + b_i)$ with $\mathbf{h} = N(\text{desc}(a, S))$.
- $\Psi_c : [0, 1]^k \rightarrow \{\text{ALLOW}, \text{DENY}\}$ is the authorization-conditioned policy. Given $A(c) \subseteq \mathcal{E}$ and thresholds $\{\theta_i\}$, $\Psi_c(\mathbf{p}) = \text{DENY} \iff \exists E_i \notin A(c) : p_i > \theta_i$.
- $\Omega : \mathcal{A} \times \mathcal{S} \rightarrow 2^{\mathcal{E}}$ returns the realized causal-effect set. The unauthorized effect set is $U(c, a, S) = \Omega(a, S) \setminus A(c)$.

The intended safety condition is $\Omega(a, S) \subseteq A(c)$. A violation occurs when $U(c, a, S) \neq \emptyset$.

3.2 Multi-Mechanism Effects and Surface-Form Fragmentation

An agent operation can be described through different *surface forms* while producing identical causal consequences.

Definition 3.1 (Multi-Mechanism Effect). *A causal effect E_i is multi-mechanism if there exist at least two distinct surface forms that can produce it:*

$$\text{MultiMech}(E_i) \iff \exists S_a \neq S_b \in \mathcal{S} : P(E_i = 1 \mid S = S_a) > 0 \wedge P(E_i = 1 \mid S = S_b) > 0.$$

The core diagnostic question is whether the LLM’s representation encodes E_i invariantly across these surface forms. We operationalize this through two metrics:

ToolProxyGap. For held-out form B , let $\hat{\tau}_{i,-B}^{\text{diag}}$ be a diagnostic probe trained on all forms except B , and $\hat{\tau}_{i,B}^{\text{within}}$ a within-form probe via stratified cross-validation on B :

$$\text{ToolProxyGap}_i(B) = [\text{FNR}_B(\hat{\tau}_{i,-B}^{\text{diag}}) - \text{FNR}_B(\hat{\tau}_{i,B}^{\text{within}})]_+.$$

This measures how much detection degrades when coverage for a form is missing, relative to how detectable the same effect is within that form.

pIIA (probe-mediated Interchange Intervention Accuracy). At layer ℓ , we inject the mean projection difference between a source sample (form B , $E_i = 1$) and a base sample (form B , $E_i = 0$) along the probe direction $\hat{\mathbf{w}}_A$ learned from form A , and evaluate whether the final-layer probe τ_i^B registers the injected effect:

$$\text{pIIA}_i(A \rightarrow B) = P(\tau_i^B(N_{>\ell}(I_{\mathbf{w}_A}(\mathbf{h}_\ell^b, \mathbf{h}_\ell^s))) > \tau_i^B(\mathbf{h}_L^b) \mid E_i(b) = 0, E_i(s) = 1).$$

pIIA is *probe-mediated*: the intervention direction and evaluation probe come from different surface forms, making it a diagnostic for cross-form effect-direction transfer, not an SCM-based causal proof.

3.3 Safety-Centric Invariance

Definition 3.2 (SafeInv). A probe τ_i is $(\epsilon_{\text{safe}}, \epsilon_{\text{fpr}})$ -safety-invariant for multi-mechanism effect E_i if:

$$\max_{S \in \mathcal{S}_{E_i}} R_{S,\theta}^-(\tau_i) \leq \epsilon_{\text{safe}} \quad \text{and} \quad \max_{S \in \mathcal{S}_{E_i}} R_{S,\theta}^+(\tau_i) \leq \epsilon_{\text{fpr}}$$

where $R_{S,\theta}^-(\tau_i) = P(\tau_i(\mathbf{h}) \leq \theta_i \mid E_i = 1, S)$. In safety-critical settings, $\epsilon_{\text{safe}} \ll \epsilon_{\text{fpr}}$.

Definition 3.3 (Auth-SafeInv). A probe τ_i is $(\epsilon_{\text{unauth}}, \epsilon_{\text{fauth}})$ -authorization-safe-invariant for effect E_i if, over task contexts c and surface forms S where E_i can be realized but is not authorized,

$$\max_{c,S} P(\tau_i(\mathbf{h}) \leq \theta_i \mid E_i \in U(c, a, S)) \leq \epsilon_{\text{unauth}},$$

and the corresponding false positive rate on authorized or absent effects is at most ϵ_{fauth} . Auth-SafeInv upgrades SafeInv by conditioning the detection target on task authorization.

3.4 EffectVerif-AuthMonitor

The verifier-assisted framework separates two questions: *which effects occurred?* and *which occurred effects were authorized?*

EFFECTVERIF-AUTHMONITOR

Input: task context c ; envelope $A(c)$; tool call (a, S, x) ; execution trace t

1. $\hat{\Omega} \leftarrow \text{VERIFYEFFECTS}(a, S, x, t)$

2. $\hat{U} \leftarrow \hat{\Omega} \setminus A(c)$

3. if $\hat{U} \neq \emptyset$ **return** DENY with violated effects $\hat{U}$

4. **else return** ALLOW

This is a framework interface, not a claim that the verifier is perfect. The trace schema includes tool name, arguments, execution status, file diffs, HTTP request/response metadata, stdout/stderr, provider status, and message receipts. The empirical question is whether adding such evidence reduces unauthorized-effect misses relative to monitors that only see task text or static policy rules.

3.5 Risk Accounting

Lemma 3.1 (Authorization-Conditioned FNR Risk Accounting). *For task context c and surface form B where action a realizes $E \notin A(c)$, define $\beta_{auth} = P(\tau_E(\mathbf{h}) \leq \theta_E \mid E \in U(c, a, B))$ and $\alpha_{auth} = P(\exists E_j \notin A(c), E_j \neq E : \tau_j(\mathbf{h}) > \theta_j \mid E \in U(c, a, B))$. Then:*

$$P(\mathcal{S}_c(a, B) = ALLOW \mid E \in U(c, a, B)) \geq [\beta_{auth} - \alpha_{auth}]_+.$$

This is a risk-accounting result, not a safety certificate. It says that if an unauthorized-effect detector has high FNR on a specific surface form, and if other detectors do not redundantly fire, then unsafe allowance is lower-bounded. The LOTO diagnostic instantiates an effect-level analogue with β^{LOTO} and α^{LOTO} .

Theorem 3.2 (Verification Sample Complexity). *Let $q = P(S = B, E \in U(c, a, B))$. To observe at least one such sample with probability $\geq 1 - \delta$ requires $m \geq \frac{1}{q} \log \frac{1}{\delta}$; to estimate FNR to accuracy $\pm \epsilon$ requires $m = \tilde{O}\left(\frac{1}{q\epsilon^2} \log \frac{1}{\delta}\right)$.*

Theorem 3.2 explains why rare tool-effect or authorization-violation combinations are difficult to validate by natural sampling alone. Contrastive projection theory (generalization bounds and alignment-FNR relationships) is deferred to the appendix; it provides conditional motivation for observed-pair repair but its assumptions do not hold in the strict held-out settings where pure representation methods fail.

4 Experiments

4.1 Setup

Data. We use several increasingly strict data regimes. The original diagnostic benchmark contains 459 agent tool-call scenarios spanning 11 causal effects and 9 tools. Each scenario includes a task context, tool call with surface form and arguments, and binary effect labels. The key design principle is breaking the deterministic tool-to-effect shortcut: the same tool produces different effects based on context, and the same effect can be produced through different tools. We also build an expanded Auth-SafeInv suite: 1,464 authorization counterfactual rows with explicit $A(c)$ envelopes, plus 35,136 schema-conditioned candidate-effect rows. For trace-based evaluation, we collect 8,988 candidate-effect rows across six trace families: controlled local sandbox observations (1,104 rows), real-agent-tools local file/terminal adapters (384 rows), a single-provider DeepSeek

Table 2: Layer 1: LOTO FNR under coverage-missing stress tests. N^+ = positive samples in heldout form. Within-FNR is heldout-form internal CV; Held-FNR trains on all non-heldout forms. ToolProxyGap = $[\Delta\text{FNR}]_+$.

Effect	Training Forms	Heldout	N^+	Within-FNR	Held-FNR	ToolProxyGap
content_fetched	all-but-terminal	terminal	8	0.111	0.500	0.389
content_fetched	all-but-web_fetch	web_fetch	32	0.064	0.781	0.718
file_content_read	all-but-read_file	read_file	32	0.033	0.875	0.842
file_deleted	all-but-terminal	terminal	20	0.246	0.600	0.354
file_written	all-but-write_file	write_file	33	0.091	0.970	0.879
network_egress	all-but-web_search	web_search	26	0.074	0.731	0.657
tool_error	all-but-web_search	web_search	10	0.278	0.100	0.000

API trace set (960 rows), broader web/search/browser/messaging local adapters (2,400 rows), file-backed headless Chrome DOM/JavaScript runtime traces (960 rows), and key-free live protocol traces combining real outbound HTTPS and local webhook protocol I/O (2,400 rows; full-precision). All trace-based rows are controlled diagnostics, not deployed-agent traffic.

Models. We extract embeddings from Qwen3-8B (4096d), with supplementary results from MiniLM (384d) and Qwen2.5-7B-Instruct (3584d). Probes are ℓ_2 -regularized logistic regression. LOTO probes train on all non-held-out forms; within-form FNR uses 3-fold stratified cross-validation inside the held-out form. pIIA uses hook-based intervention at layers 24/36 of Qwen3-8B. Thresholds in trace-based evaluation are selected on validation trace groups and evaluated on held-out trace groups.

Metrics. We report FNR (the safety-critical metric), FPR, ToolProxyGap, and pIIA-Drop ($\text{pIIA}_{\text{within}} - \text{pIIA}_{\text{cross}}$). For action-level evaluation, we report Unauthorized Action Allow Rate (U-Allow) and Authorized Action False Denial Rate (FDeny).

4.2 Layer 1: Tool-Surface Fragmentation

The first layer of over-optimism is evaluating effect detectors only on seen tool surfaces.

LOTO stress test. Table 2 shows leave-one-tool-out results on Qwen3-8B. Within-form FNR ranges from 0.03 to 0.48, confirming that within-form detection is feasible but not uniformly trivial. Held-out FNR can be extreme: 0.97 for `file_written/write_file`, 0.88 for `file_content_read/read_file`, 0.78 for `content_fetched/web_fetch`. The stress-test signal is the *degradation*—ToolProxyGap reaches 0.88 for `file_content_read` and 0.72 for `content_fetched` on its primary tool. A monitor evaluated only within its training tools would see low FNR; deployed on a held-out tool it can miss most unauthorized effects.

pIIA activation-level evidence. Table 3 provides convergent diagnostic evidence. `tool_error`—whose cross-form semantics (“failure/denied/blocked”)

Table 3: pIIA results (Qwen3-8B, layer 24/36). 95% CIs are source-cluster bootstrap.

Effect	pIIA-within	pIIA-cross	pIIA-Drop	95% CI
tool_error	1.000	0.948	0.052	[0.032, 0.077]
file_content_read	1.000	0.837	0.164	[0.050, 0.286]
file_deleted	1.000	0.800	0.200	[0.094, 0.325]
content_fetched	1.000	0.702	0.298	[0.143, 0.544]
file_written	1.000	0.669	0.331	[0.147, 0.550]
network_egress	1.000	0.634	0.366	[0.307, 0.424]

are linguistically unified—achieves near-perfect cross-form transfer (pIIA-Drop = 0.052). `content_fetched`—whose realization through `web_fetch` (“fetched JSON from API”) and `terminal` (“curl -s URL”) involves disjoint linguistic patterns—shows larger cross-form drop (pIIA-Drop = 0.298). pIIA is probe-mediated and is used as a diagnostic, not a causal proof.

Not a lexical shortcut. Lexical normalization (replacing tool names with [TOOL], inline code with [CMD], URLs with [URL]) leaves overall held-out FNR essentially unchanged: $0.378 \rightarrow 0.382$ across matched cells. Individual cells move in both directions (`file_written` $0.97 \rightarrow 0.70$; `network_egress/terminal` unchanged at 0.41). The fragmentation persists in the representation.

No simple baseline eliminates the failure. Pooled training, balanced subsampling, tool-conditioned probes, group reweighting, and Procrustes alignment all leave residual worst-form FNR. Balanced pooling helps on some effects (`file_deleted`: $0.60 \rightarrow 0.20$) but not others (`content_fetched`: $0.78 \rightarrow 0.78$). Procrustes alignment improves only 15/38 form pairs. Sampling imbalance explains part but not all of the failure.

Coverage-missing safety bound. The LOTO stress-test lower bound $[\beta^{\text{LOTO}} - \alpha^{\text{LOTO}}]_+$ identifies cases where high held-out FNR translates to a positive unsafe-allow bound. The largest bound is `file_content_read/read_file`: $\beta^{\text{LOTO}} = 0.88$, $\alpha^{\text{LOTO}} = 0.19 \Rightarrow [\beta - \alpha]_+ = 0.69$. Some high-FNR cells have zero bound because of sufficient predicted redundancy (`content_fetched/terminal`: $\beta = 0.50$, $\alpha = 1.00$). Both high-FNR and low-redundancy must coincide for a safety vulnerability. See appendix for the full table.

4.3 Layer 2: Trace-Label Over-Optimism

The second layer of over-optimism is evaluating monitors on structured trace labels that are close to the answer. Table 4 reports a trace-view ablation: the same validation-selected threshold policy is evaluated with full label fields, with label fields hidden, and with only minimal low-level execution evidence.

Full-label FNR of 0.0353 would suggest strong performance. But removing label fields increases mean FNR by $3.3\times$ to 0.1162, and further reducing to minimal evidence increases it by $4.2\times$ to 0.1465. The degradation is largest

Table 4: Layer 2: Trace-view verifier ablation (mean across T58/T59/T61/T63/T65/T64 held-out trace groups). Label-hidden hides `verified_effects`, `effect_diff`, `unauthorized_effects`, and related fields. Minimal-evidence keeps only file diffs, exit codes, HTTP metadata, `stdout/stderr`.

View	Mean row FNR	Mean row FPR	Mean U-Allow	Mean FDeny
Full label (upper bound)	0.0353	0.0000	0.0417	0.4544
Label-hidden raw	0.1162	0.0480	0.0459	0.7428
Minimal evidence	0.1465	0.0643	0.0531	0.8417

Table 5: Layer 3: Action-level metrics from the same T62 validation-selected thresholds. U-Allow = Unauthorized Action Allow Rate; FDeny = Authorized Action False Denial Rate. Execution verifier can achieve row-level FNR/FPR of 0/0 on some traces while having action-level FDeny of 1.0.

Trace family	Exec U-Allow	Exec FDeny	Static U-Allow	Static FDeny
Controlled auth traces	0.0000	0.1591	0.0000	0.4091
real-agent-tools local adapters	0.2000	0.3571	0.2000	0.3571
DeepSeek provider API	0.0000	1.0000	1.0000	0.0000
Broader local adapters	0.0500	0.2100	0.4875	0.2100
Headless Chrome runtime	0.0000	1.0000	1.0000	0.0000
Live HTTP + webhook protocol	0.0000	0.0000	0.3210	0.0000

on the file-backed browser-runtime traces, where the static verifier already fails completely (FNR 1.0). Action-level false denial also increases substantially. This does not mean the verifier is purely reading labels—the label-hidden view still outperforms pre-action rule-only baselines (mean FNR 0.57, see Layer 3 below)—but it means that full-label results should be treated as upper bounds, not as evidence of deployment-ready monitoring. The current verifier depends on structured execution evidence.

4.4 Layer 3: Row-to-Action Metric Translation

The third layer of over-optimism is the gap between candidate-effect row metrics and action-level allow/deny decisions.

Row-level success does not imply action-level safety. Table 5 aggregates validation-selected candidate-effect predictions into action-level metrics. A real monitor ultimately decides whether an action should be allowed; it is not sufficient to report that individual effect predictions have low error rates.

The execution verifier achieves row-level FNR/FPR of 0.0/0.0 on the provider API and browser-runtime traces, yet action-level false denial rates reach 1.0 on both. This happens because an action is denied if *any* present candidate effect is scored as unauthorized; even a single false positive on an authorized effect causes the whole action to be rejected. The gap between row-level and

Table 6: Three gate-failure results. All values are at $\text{FPR} \leq 0.10$ under strict held-out splits. Best baseline is the strongest same-cell non-degenerate baseline.

Method	Setting	FNR (method)	FNR (best baseline)
Contrastive projection (T54)	LOTO, 21 cells	0.2804	0.1835 (supervised contrastive)
Schema-conditioned monitor (T55)	LOTO, full-tool-chain	0.4516	0.1835 (supervised contrastive)
Decomposed frozen verifier (T56)	LOTO, full-tool-chain	0.3547	0.1835 (supervised contrastive)

action-level metrics is not a minor calibration issue—it is a structural property of how candidate-effect predictions compose into decisions.

Simple calibration does not fix this. An action-level calibration pass that constrains validation false denial to ≤ 0.10 often selects a degenerate all-allow threshold. On the real-agent-tools local adapters, broader local adapters, provider API, and browser-runtime trace families, the calibration chooses thresholds that allow all actions, yielding $\text{U-Allow} = 1.0$. This means the constraint of low false denial cannot be met without effectively disabling the monitor, exposing a fundamental tension between safety (blocking unauthorized actions) and utility (not blocking authorized ones) in current effect-based monitoring.

4.5 Gate Failure: Pure Representation Repair

Before turning to the constructive framework, we report an important negative result: three attempts at pure representation-side repair—methods that only modify the LLM’s embedding space without external execution evidence—all fail to beat strong baselines under strict held-out protocols.

Table 6 summarizes the three gate failures. Contrastive projection achieves 21/21 evaluable LOTO cells after a data expansion (up from 4/14) but its strict train-only FNR of 0.2804 is substantially worse than the strongest same-cell baseline (supervised contrastive, $\text{FNR} = 0.1835$). A pair-free schema-conditioned monitor—which asks the model to infer effects from tool descriptions without seeing cross-form pairs—does worse still ($\text{FNR} = 0.4516$). Decomposing the task into separate effect-presence and authorization probes, using only frozen Qwen3-8B embeddings, does not close the gap ($\text{FNR} = 0.3547$).

These are not null results in the usual sense—they are *gate* failures: each method makes a different structural assumption about why the problem should be solvable from representations alone, and each fails the strict test. We report them because they establish the difficulty of the problem and prevent overclaiming about representation-side solutions.

Verifier-present upper bound. When an oracle label indicates which effects are actually present, a simple authorization classifier achieves LOTO $\text{FNR} = 0.0042$ and family $\text{FNR} = 0.0745$ at $\text{FPR} = 0$. This is an upper bound for the two-stage architecture (verify effects, then check authorization)—but it uses an oracle, not a real verifier. The constructive question is whether a non-oracle verifier can approach this bound.

Table 7: EffectVerif-AuthMonitor results. Thresholds selected on validation trace groups, evaluated on held-out trace groups. Static verifier uses tool-call rules without trace evidence. These are controlled diagnostics, not deployment calibration.

Setting	Trace family	Rows	Exec verifier FNR/FPR	Static verifier FNR/FPR
T58/T62	Controlled auth traces	1,104	0.0000 / 0.0000	0.0000 / 0.0520
T59/T62	real-agent-tools local adapters	384	0.1765 / 0.0000	0.1765 / 0.0000
T61/T62	DeepSeek provider API	960	0.0000 / 0.0000	1.0000 / 0.0000
T63	Broader local adapters	2,400	0.0351 / 0.0000	0.6140 / 0.0132
T65	Headless Chrome runtime	960	0.0000 / 0.0000	1.0000 / 0.0000
T64	Live HTTP + webhook protocol	2,400	0.0000 / 0.0000	0.6294 / 0.0462

4.6 Constructive Framework: EffectVerif-AuthMonitor

We evaluate the EffectVerif-AuthMonitor framework (Section 3.4) by replacing the oracle present label with deterministic execution-trace and static verifiers, then measuring whether the authorization monitor can detect unauthorized effects on held-out trace groups.

Table 7 shows a consistent pattern: execution-trace verifiers substantially outperform static verifiers, especially on provider, broader-tool, browser-runtime, and protocol effects. On the live protocol traces, the execution verifier reaches test FNR/FPR of 0.0/0.0 ($N^+ = 197$, $N^- = 1170$) while the static verifier has FNR/FPR of 0.629/0.046. On the headless Chrome browser-runtime traces, execution verification reaches 0.0/0.0 ($N^+ = 62$, $N^- = 411$) while static rules miss all browser effects (FNR 1.0). Static verifiers fail because they rely on tool-name-to-effect mappings; when the same effect is produced through a tool whose name does not trigger the rule, or when the effect occurs remotely (provider API), the static rule has no signal.

Honest limits. Three important caveats apply. (1) The execution verifier is deterministic and rule-based, not an independently learned model; it was designed for these controlled trace formats and may not transfer. (2) The live protocol traces cover real outbound HTTPS and local webhook I/O, but not provider-backed search, SaaS messaging, or HTTP browser automation. The browser-runtime traces use actual headless Chrome but over file-backed DOM/JavaScript pages, not live web browsing. (3) As Layer 3 showed, row-level success does not translate to action-level safety, and simple calibration fails.

Contrastive projection as observed-pair upper bound. Full-training contrastive projection, which uses cross-form positive pairs from all tools, achieves post-FNR of 0.00 on 5/6 effects and reduces the worst case (`content_fetched`) from 0.78 to 0.25. A strict leave-one-pair-out protocol improves 40/68 tool-case evaluations (mean $\Delta\text{FNR} = +0.157$). These results confirm that the fragmentation is at least partially representation-geometric and can be repaired when cross-form pairs are observed. However, since strict train-only projection does not beat strong baselines (Section 4.5), we use contrastive projection only as evidence that representation alignment can repair observed surface pairs. Full contrastive tables, multiseed results, and alignment theory are in the appendix.

Table 8: Proxy defense comparison (mean across all held-out trace groups). Pre-action rule-only denies based on tool name and policy rules without trace evidence. Provenance-only checks whether the call context supports the action. Raw-status boundary checks exit codes and explicit error status strings.

Monitor	Mean row FNR	Mean row FPR	Mean U-Allow	Mean FDeny
Pre-action rule-only	0.5700	0.0186	0.5014	0.1627
Provenance-only	0.2873	0.0733	0.0486	0.3762
EffectVerif (label-hidden)	0.1162	0.0480	0.0459	0.7428
Raw-status boundary	0.1113	0.0101	0.0171	0.0265

4.7 Existing-Defense Proxy Comparison

Table 8 compares EffectVerif-AuthMonitor against proxy monitors that approximate existing defense paradigms without faithfully reimplementing them.

Pre-action rule-only and provenance-only monitors miss many realized-effect violations (FNR 0.57 and 0.29). EffectVerif with label-hidden evidence improves substantially on these. However, a simple handcrafted raw-status boundary—which checks only exit codes and explicit error status strings—is competitive (row FNR 0.1113, U-Allow 0.0171, FDeny 0.0265). This has two implications. First, it confirms that effect evidence helps relative to pre-action rules. Second, it means we should not claim that a learned authorization monitor *dominates* all handcrafted status-policy defenses. The contribution is the Auth-SafeInv evaluation framework and the three-layer diagnosis, not a universal claim of method superiority.

5 Discussion

Our results paint a consistent picture: agent safety monitoring evaluation can be systematically over-optimistic, and the over-optimism operates at distinct layers that require distinct diagnostic methods. We draw four broader implications.

The evaluation target matters. When safety is evaluated as attack success rate, task completion, or pooled F1, the specific failure mode of unauthorized-effect misses under coverage gaps is invisible. Auth-SafeInv makes this failure mode explicit by conditioning the target on task authorization and requiring held-out surface-form evaluation. The field needs evaluation targets that match the safety guarantee being claimed.

Each layer hides the next. The three layers we identify are not independent. A monitor that appears safe at Layer 1 (low within-tool FNR) can fail at Layer 2 (the low FNR depends on structured labels). A monitor that passes Layer 2 (reasonable label-hidden FNR) can fail at Layer 3 (row-level metrics do not translate to safe action policies). Evaluation at only one layer can therefore be misleading. We recommend that agent safety monitor evaluations report at minimum: (a) held-out tool or trace-family FNR/FPR, (b) a label-field ablation

comparing full and minimal trace views, and (c) action-level U-Allow and FDeny alongside row-level metrics.

Negative results are evidence. The three gate failures (T54/T55/T56) are not null results—they are evidence that pure representation-side repair, at least in the forms we tested, is insufficient for this problem. Reporting them prevents the literature from retrying approaches that fail under strict evaluation. We encourage the field to report such gate failures explicitly, with clear protocols and strong baselines, so that method development targets the right difficulty level.

Constructive frameworks need honest limits. EffectVerif-AuthMonitor is not a deployed safety solution. It works under controlled settings, depends on structured execution evidence, and has unresolved action-level calibration. But it serves as a constructive baseline: a concrete instantiation of the two-stage architecture (verify effects, then check authorization) against which future methods can be compared. The framework’s limits—static verifier failures on provider/browser effects, competitive raw-status baselines, all-allow calibration collapse—also define the research agenda.

6 Limitations

1. **Data.** The original diagnostic uses synthetic scenarios; lexical artifacts remain a risk despite controls. Auth-SafeInv traces are controlled (local sandbox, local adapters, single-provider API, file-backed browser, key-free live protocol) and do not cover provider-backed search, SaaS messaging, HTTP browser automation, or deployed-agent runtime logs.
2. **Model scale.** All experiments use models $\leq 8\text{B}$ parameters. Larger-model behavior is unknown.
3. **Surface-form coverage.** Our fragmentation diagnosis is restricted mainly to tool switching ($\mathcal{S} = \mathcal{T}$). Extensions to context injection, semantic reframing, and language variation require separate validation.
4. **Verifier.** The execution verifier is deterministic and rule-based. Independent learned or formally specified effect verifiers are needed for deployment-facing claims.
5. **Action-level policy.** We show that simple threshold calibration fails (T73), but do not provide a working solution. Abstention policies, utility-aware calibration, and human-in-the-loop designs remain future work.
6. **External defense comparison.** Our proxy baselines are conceptual approximations, not faithful reimplementations of external systems.
7. **Statistical.** Some FNR estimates have small positive counts and wide confidence intervals. We report Wilson CIs in the appendix and recommend cautious interpretation for cells with $N^+ < 30$.

7 Reproducibility

All experiments are implemented in Python with PyTorch and scikit-learn. The original diagnostic pipeline is automated via `run_experiments.sh`. The Auth-SafeInv pipeline is tracked by `reproduce_auth_safeinv.sh` and a result-source appendix mapping each reported artifact to its script, command, output files, and caveats. API-key-dependent provider traces are recorded as artifacts but not rerun by default. Code, data, and analysis outputs will be released as an open-source repository upon publication.

References

- [1] *JAW: Comment and Control—Hijacking Agentic Workflows via Context-Grounded Evolution*. arXiv:2605.11229, May 2026.
- [2] *LITMUS: Benchmarking Behavioral Jailbreaks of LLM Agents in Real OS Environments*. arXiv:2605.10779, May 2026.
- [3] *AttriGuard: Defeating Indirect Prompt Injection via Causal Attribution of Tool Invocations*. arXiv:2603.10749, Mar 2026.
- [4] *ClawGuard: A Runtime Security Framework for Tool-Augmented LLM Agents*. arXiv:2604.11790, Apr 2026.
- [5] *ARGUS: Defending LLM Agents Against Context-Aware Prompt Injection*. arXiv:2605.03378, May 2026.
- [6] *XOA: Execute-Only Agents*. Agentic OS Workshop, 2026.
- [7] *PACT: The Granularity Mismatch in Agent Security: Argument-Level Provenance Solves Enforcement and Isolates the LLM Reasoning Bottleneck*. arXiv:2605.11039, May 2026.
- [8] *Alignment Contracts for Agentic Security Systems*. arXiv:2605.00081, Apr 2026.
- [9] *ARM: Causality Laundering—Denial-Feedback Leakage in Tool-Calling LLM Agents*. arXiv:2604.04035, Apr 2026.
- [10] *AgentTrust: Runtime Safety Evaluation and Interception for AI Agent Tool Use*. arXiv:2605.04785, May 2026.
- [11] *Parallax: Why AI Agents That Think Must Never Act*. arXiv:2604.12986, Apr 2026.
- [12] *ECA: Hallucination as Exploit—Evidence-Carrying Multimodal Agents*. arXiv:2605.19192, May 2026.
- [13] *A Systematic Survey of Security Threats and Defenses in LLM-Based AI Agents*. arXiv:2604.23338, Apr 2026.

- [14] *SoK: The Attack Surface of Agentic AI*. arXiv:2603.22928, Mar 2026.
- [15] *QueryIPI: Query-agnostic Indirect Prompt Injection on Coding Agents*. arXiv:2510.23675, Jan 2026.
- [16] *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents*. NeurIPS Datasets and Benchmarks, 2024.
- [17] *Identifying the Risks of LM Agents with an LM-Emulated Sandbox*. ICLR, 2024.
- [18] *AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents*. ICLR, 2025.
- [19] *Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents*. ICLR, 2025.
- [20] *ST-WebAgentBench: A Benchmark for Evaluating Safety and Trustworthiness in Web Agents*. ICLR, 2026.
- [21] *TraceSafe: A Systematic Assessment of LLM Guardrails on Multi-Step Tool-Calling Trajectories*. arXiv:2604.07223, Apr 2026.
- [22] *CausalArmor: Efficient Indirect Prompt Injection Guardrails via Causal Attribution*. arXiv:2602.07918, Feb 2026.
- [23] M. Arjovsky, L. Bottou, I. Gulrajani, and D. Lopez-Paz. *Invariant Risk Minimization*. arXiv:1907.02893, 2019.
- [24] S. Sagawa, P. W. Koh, T. B. Hashimoto, and P. Liang. *Distributionally Robust Neural Networks for Group Shifts*. ICLR, 2020.
- [25] A. Geiger et al. *Causal Abstractions of Neural Networks*. NeurIPS, 2021.
- [26] A. Geiger et al. *Inducing Causal Structure for Interpretable Neural Networks*. ICML, 2022.
- [27] A. Geiger et al. *Causal Abstraction: A Theoretical Foundation for Mechanistic Interpretability*. JMLR, 2025.
- [28] T. Sutter et al. *The Non-Linear Representation Dilemma*. NeurIPS, 2025.
- [29] J. Zhou et al. *From Small to Large: In-Context Learning for Domain Generalization*. IJCV, 2026.
- [30] Y. Bao et al. *Faithful Bi-Directional Model Steering via Distribution Matching and DII*. ICLR, 2026.